

Study guide for Exam GH-200: GitHub Actions

Purpose of this document

This study guide should help you understand what to expect on the exam and includes a summary of the topics the exam might cover and links to additional resources. The information and materials in this document should help you focus your studies as you prepare for the exam.

 Expand table

Useful links	Description
How to earn the certification	Some certifications only require passing one exam, while others require passing multiple exams.
Certification renewal	Microsoft associate, expert, and specialty certifications expire annually. You can renew by passing a free online assessment on Microsoft Learn.
Your Microsoft Learn profile	Connecting your certification profile to Microsoft Learn allows you to schedule and renew exams and share and print certificates.
Exam scoring and score reports	A score of 700 or greater is required to pass.
Exam sandbox 	You can explore the exam environment by visiting our exam sandbox.
Request accommodations	If you use assistive devices, require extra time, or need modification to any part of the exam experience, you can request an accommodation.

About the exam

Some exams are localized into other languages, and those are updated approximately eight weeks after the English version is updated. While Microsoft makes every effort to update localized versions as noted, there may be times when the localized versions of an exam are not updated on this schedule. Other available languages are listed in the **Schedule Exam** section of the **Exam Details** webpage. If the exam isn't available in your preferred language, you can request an additional 30 minutes to complete the exam.

Note

The bullets that follow each of the skills measured are intended to illustrate how we are assessing that skill. Related topics may be covered in the exam.

Note

Most questions cover features that are general availability (GA). The exam may contain questions on Preview features if those features are commonly used.

Skills measured as of January 2026

Audience profile

Candidates for this exam should have expertise in automating software development workflows with GitHub Actions. This includes creating and maintaining workflows and actions, managing GitHub Actions at scale, and ensuring secure and efficient automation for organizations and enterprises. Candidates should also be familiar with CI/CD, GitHub repositories, GitHub Packages, and integrating third-party services.

Skills at a glance

- Author and manage workflows (20–25%)
- Consume and troubleshoot workflows (15–20%)
- Author and maintain actions (15–20%)
- Manage GitHub Actions for the enterprise (20–25%)
- Secure and optimize automation (10–15%)

Author and manage workflows (20–25%)

Configure workflow triggers and events

- Configure workflows to run for scheduled, manual, webhook, and repository events
- Choose appropriate scope, permissions, and events for workflow automation
- Define and validate workflow_dispatch inputs (types, required, defaults) and pass inputs to reusable workflows via workflow_call with inputs and secrets mapping

Design and implement workflow structure

- Use jobs, steps, and conditional logic
- Implement dependencies between jobs
- Use workflow commands and environment variables
- Use service containers (services:) for dependent services (databases, queues); configure ports, health checks, and container options
- Use strategy and matrix to generate job variations (OS, language/runtime versions); apply include/exclude; control fail-fast and max-parallel; optimize matrix size for cost and performance; account for runner image changes (Ubuntu 20.04 deprecation, Windows Server 2025 migration for windows-latest)
- Implement YAML anchors and aliases (&, * and merge <<) to reuse repeated mappings/steps within a single workflow file
- Use predefined contexts (github, runner, env, vars, secrets, inputs, matrix, needs, strategy, job, steps, github.event, github.ref) to access workflow, repository, and runtime metadata; understand immutable actions behavior and version pinning requirements
- Evaluate expressions with `${{ }}` referencing contexts; distinguish static (workflow parse) vs runtime evaluation; prevent secret leakage in logs and expressions
- Leverage editor tooling (GitHub Actions VS Code extension / YAML schema completion, metadata IntelliSense, validation) to author and maintain workflows efficiently

Manage workflow execution and outputs

- Configure caching and artifact management; apply retention policies via REST APIs (logs, artifacts, workflow runs) at org/repo level
- Pass data between jobs and steps (artifacts, outputs, environment files via `GITHUB_ENV` and `GITHUB_OUTPUT`, reusable workflow outputs)
- Generate job summaries using `GITHUB_STEP_SUMMARY` for rich Markdown reports (test results, coverage, links)
- Add workflow status badges and environment protections

Consume and troubleshoot workflows (15–20%)

Interpret workflow behavior and results

- Identify workflow triggers and effects from configuration and logs
- Diagnose failed workflow runs using logs and run history
- Expand and interpret YAML anchors, aliases, and merged mappings when analyzing workflow configuration
- Interpret matrix expansions, correlate job names to matrix axes, analyze failures across variants, and selectively rerun individual matrix jobs

Access workflow artifacts and logs

- Locate workflows, logs, and artifacts in the UI and via API
- Download and manage workflow artifacts

Use and manage workflow templates

- Consume organization-level and reusable workflows
- Consume non-public organization workflow templates
- Use starter workflows (public and private/non-public templates); customize and adapt; distinguish from reusable workflows and composite actions
- Differentiate starter workflows (copy scaffold, independent after creation) vs reusable workflows (central versioned definition invoked via `workflow_call`) vs composite actions (encapsulated step logic)
- Contrast disabling and deleting workflows

Author and maintain actions (15–20%)

Create and troubleshoot custom actions

- Identify and implement action types (JavaScript, Docker, composite); understand immutable actions rollout on hosted runners and implications for version pinning and registry sources
- Troubleshoot action execution and errors

Define action structure and metadata

- Specify required files, directory structure, and metadata

- Implement workflow commands within actions

Distribute and maintain actions

- Select distribution models (public, private, marketplace)
- Publish actions to the GitHub Marketplace
- Apply versioning and release strategies

Manage GitHub Actions for the enterprise (20–25%)

Distribute and govern actions and workflows

- Define and manage reusable components and templates
- Control access to actions and workflows within the enterprise
- Configure organizational use policies

Manage runners at scale

- Configure and monitor GitHub-hosted and self-hosted runners
- Apply IP allow lists and networking settings
- Manage runner groups and troubleshoot runner issues
- Identify preinstalled software/tool versions on GitHub-hosted runners (image release notes, toolcache) and install additional software at runtime (setup-* actions, package managers, caching, container images, custom self-hosted images)

Manage encrypted secrets and variables

- Define and scope encrypted secrets and variables at the organization, repository, and environment levels
- Access and use secrets and variables in workflows and actions; manage secrets and variables programmatically via REST APIs

Secure and optimize automation (10–15%)

Implement security best practices

- Use environment protections and approval gates
- Identify and use trustworthy actions from the Marketplace
- Mitigate script injection (sanitize/validate inputs, least-privilege permissions, avoid untrusted data in run; proper shell quoting, prefer vetted actions over inline scripts)
- Understand GITHUB_TOKEN lifecycle (ephemeral, scoped), configure granular permissions, contrast with PAT; restrict write scopes
- Use OIDC token (id-token permission) for cloud provider federation to eliminate long-lived cloud secrets
- Pin third-party actions to full commit SHAs; align with immutable actions enforcement on hosted runners; avoid floating @main/@v* without justification
- Enforce action usage policies (organization/repository allow/deny lists, required reviewers for unverified actions)
- Generate and verify artifact attestations / provenance (e.g., SLSA, build metadata) and integrate into deployment verification

Optimize workflow performance and cost

- Configure caching and artifact retention for efficiency; apply retention policies programmatically via REST APIs
- Recommend strategies for scaling and optimizing workflows

Study resources

We recommend that you train and get hands-on experience before you take the exam. We offer self-study options and classroom training as well as links to documentation, community sites, and videos.

 Expand table

Study resources	Links to learning and documentation
Get trained	Choose from self-paced learning paths and modules or take an instructor-led course on Microsoft Learn – Automate your workflow with GitHub Actions

Study resources	Links to learning and documentation
Find documentation	Author and maintain workflows  Consume workflows  Author, release, and maintain actions  Manage GitHub Actions for the enterprise
Ask a question	GitHub Community Discussions 
Get community support	GitHub Blog 
Follow GitHub	Twitter  LinkedIn  Instagram 
Find a video	YouTube 

Change log

This exam has changed significantly (e.g., new objectives were added, some were removed, existing objectives may have moved to different functional groups, and all were reworded) on January, 2026.

Last updated on 02/05/2026